

TableClaw

0. 一句话定位

TableClaw 是基于 nanobot 二次开发的「表格专精 Agent」，覆盖 4 大场景：QA / 编辑修复清洗 / 表格转换跨表合并 / 报表生成与从 0 建表。
当前完成原型 + 双轨迹 mentor demo + 10 任务评测，已具备一键启动 / 一键评测 / 一键 demo 三套入口。

1. 项目结构速览

TableClaw/	
├─ start.sh / eval.sh / demo.sh	# 一键启动 / 一键评测 / 一键 mentor demo
├─ nanobot/	# 上游框架本体（裁剪后保留）
│ └─ nanobot/	
│ └─ agent/	loop, runner, memory, context, skills, tools,
autocompact	
│ └─ channels/	多平台适配 (Telegram, WeChat, Slack, ...)
│ └─ providers/	多 LLM 接入 (Anthropic, OpenAI, DashScope, Bedrock, ...)
│ └─ skills/	内置 skill: xlsx, tc-bigtable-header, tc-bigtable-
aggregate, ...	
│ └─ session/	会话历史 + TTL 自动压缩
│ └─ api/	OpenAI 兼容 HTTP API
│ └─ webui/	Vite + React SPA (未启用)
├─ workspace/	# 运行态: memory / sessions / usage /
用户级 skill	
├─ eval_test/	# 评测数据 + 两条 runner (skill_matrix
+ mentor_demo)	
├─ docs/	# 项目文档 (架构 / 功能开发 / 实验评测 /
项目管理)	
├─ skills/	# 三家参考 skill (codex / kimi /
anthropic)	
└─ test_table/	# 原始工业表格池

各部分一句话职责

部分	一句话职责
nanobot/	Agent loop + 多 channel + 多 provider + 工具调用, TableClaw 的基座
nanobot/nanobot/skills/xlsx	当前表格主 skill (codex 原文, 待替换为 TableClaw 自研)
nanobot/nanobot/skills/tc-bigtable-*	mentor demo 用的两个针对宽表的小 skill
workspace/	nanobot 运行态: memory / sessions / 用户级 skill / token usage log
eval_test/	评测数据集 + 两条 runner (10 任务主线 + 1 复合 demo)
docs/	文档体系: 架构 / 功能开发 / 实验评测 / 项目管理 (含 TODO + dev-log)
skills/	三家外部参考 skill (codex / kimi / anthropic), 不直接挂载
test_table/	~100 张工业表, 原始池

2. 已做 (截至 2026-06-01)

2.1 基础设施

- 跑通 nanobot + 百炼 DashScope (deepseek-v4-pro, reasoningEffort=high)
- 一键启动 `./start.sh`、一键评测 `./eval.sh`、一键 demo `./demo.sh`
- Workspace 迁到项目内, 便于查 USER/SOUL/AGENTS/memory/sessions
- 运行时 token usage 持久化到 `workspace/usage/usage.jsonl` (filelock 保护)

2.2 Skill 体系 (builtin)

Skill	来源	用途	体量
xlsx	codex 原文	skill_matrix 主线	1068 行

tc-bigtable-header	自研	mentor demo：多级表头展开	≤80 行
tc-bigtable-aggregate	自研	mentor demo：按层级父级聚合	≤80 行

2.3 评测体系

两条线并行，不互相串：

实验线	Runner	Dataset	报告
Skill Matrix	<code>./eval.sh</code>	10 任务 (simple/medium/hard)	<code>docs/实验评测/skill-matrix/</code>
Mentor Demo	<code>./demo.sh</code>	1 复合任务（欠费表 228×318）	<code>docs/实验评测/mentor-demo/</code>

- skill_matrix 验证 codex skill 在不同难度下的边际价值（结论：skill-on auto pass 8/10 vs 7/10, token 多 ~2.5%；提示 codex 原文偏重）
- mentor_demo 演示 agent 在不同步骤调用不同小 skill 的"分工"画面（5 步 / 56,894 tokens vs 4 步 / 63,172 tokens, skill-on 省 ~10%）
- 每次 `./demo.sh` 自动按时间戳归档到 `eval_test/results/mentor_demo/runs/<ts>/`，不覆盖历史

2.4 工程交付

- `.gitignore` 就绪（覆盖 venv / 运行态 / latest 产物 / 归档目录）
- 文档体系四级：架构 / 功能开发 / 实验评测 / 项目管理；每级有 README 索引
- TODO 计划 + 开发日志双文档分工

3. 当前最关键的判断与限制

讲给 mentor 听的 3 个"真话"：

3.1 现有 skill 体系的边际价值有限

- 在我们的强基模（deepseek-v4-pro reasoning=high）下，模型已具备相当强的表格常识，SKILL.md 形态的"指令文档"边际价值越来越小。
- 证据：codex 原文 1068 行 skill 在 10 任务上只被读 3/10，且 token 反而多 2.5%。

3.2 "agent 调用不同 skill" 的画面好看但真实必要性偏弱

- mentor demo 第二版（欠费宽表）勉强能看出 skill-on 省 ~10% token，但差距不悬殊。
- 真正能提升效率的技术而言的目标，比如下流成本优化，模型一次调用聚合结果，不用每次 read file，更

- 真正能让 Skill 住技不回赢的是把 Skill 下沉成 **tool**：模型一次调用拿结果，不用每次 read_file + 与 Python 试错。

3.3 评测体系不够支撑商用论证

- 11 题（10 主线 + 1 demo）、单一表族（四川经营指标）。
- 评分器有精度 bug（4 位小数被 1e-6 tolerance 砍掉）。
- 没有 QA 之外的场景题（编辑 / 转换 / 报表）。

4. 后续路线（按可插拔层 → 系统层划分）

4.1 可插拔层：评测集 / Tool / Skill（已在 TODO，落地路径清晰）

方向	短期目标	阻塞点
Eval Dataset	11 题 → 80-150 题，4 场景 × 3 难度 × 3 表结构复杂度	需要招同学贡献，缺标注规范
专用 Tool	实现 5 个 tableclaw_* tool，沉淀 openpyxl 套路	需先写 RFC + 命名规范
多家 Skill 横评	codex / kimi / anthropic / claude-code / 自研 5 家在同一 dataset 上横比	kimi 需 docker 跑 linux ELF； claude-code skill 原文需获取

详细见 [TODO.md](#) 中期段。

4.2 系统层：nanobot 本体改造方向（mentor 关心的重点）

这是 TableClaw 与"普通 nanobot + 提示词"的真正分水岭。下面 5 个子系统是 nanobot 原生实现的薄弱点（在表格场景下），可改造为表格适配版。

A. Memory：从"对话历史压缩"升级为"表格语义记忆"

- 现状（nanobot/nanobot/agent/memory.py）：基于 jsonl 顺序写入对话历史，配合 Dream 两阶段记忆做 summarize；按 token 上限触发压缩；workspace 下放 `MEMORY.md` 文本作为长期记忆。
- 限制：
 - 完全文本驱动，压缩等于"扔历史 + 写一段摘要"，表格的列名 / sheet 拓扑 / 已发现的合计行 / 已纠正的字段类型 全部丢失。
 - 长会话中模型会反复重新 inspect 同一张表，token 暴涨。
- 改造方向：
 - Table aware memory/schema：除了文本摘要，记结构化的 table fact 记忆

- **table-aware memory schema**：陈丁义平摘要，记结构化的 table-fact 记忆
(`{table_path, sheet, header_levels, total_rows, blank_rows, col_aliases, last_inspected_at, fingerprint(sha)}`)。
- 持久化方案二选一：jsonl + 简单索引文件（最小改动，沿用 nanobot 现有持久化习惯）；或者 sqlite（查询灵活但引入新依赖）。优先做前者，按需升级。
- 在 ContextBuilder 拼 prompt 时，如果模型即将操作某张表，自动把对应 **table-fact** 注入 **system** 段，模型无需再 inspect。

B. Context Builder：从"线性塞"升级为"按相关性拼"

- 现状 (`nanobot/nanobot/agent/context.py` + `templates/agent/skills_section.md`)：每轮把 system prompt + skill summary + recent messages 顺序拼接；skill 用"读 SKILL.md 摘要"策略。
- 限制：
 - 没有 retrieval 概念，所有 skill summary 都塞进每轮 prompt。
 - 多表场景下，所有表元数据都得塞进来或都不塞，没有"按问题相关性挑"的能力。
- 改造方向：
 - **Skill retrieval**（需新增 **embedding** 依赖）：用 embedding 把 skill description 索引化，按用户问题召回 top-k skill 注入 active 段（而不是让模型自己从摘要里挑）。embedding model 当前 nanobot 未集成，先评估用 **dashscope/openai 现成 embedding API** 还是本地 **sentence-transformers**，再动手。
 - **Table retrieval**（见 4.3）：同一思路用在表上。
 - 分层 **system prompt**：核心规则 + 任务相关上下文 + 召回结果，每层独立 cache。

C. Autocompact / Session：TTL 压缩对表格无感

- 现状 (`nanobot/nanobot/agent/autocompact.py` + `nanobot/nanobot/session/manager.py`)：按 token 上限 + TTL 自动 summarize 旧消息。
- 限制：压缩规则与表格语义无关——可能把"已确认表头结构"这类高复用信息压掉，把闲聊保留。
- 改造方向：
 - 压缩规则区分事实层（结构、列名、口径）与对话层（寒暄、确认）。
 - 事实层压缩前先抽到 **table-fact memory** (4.A)，保证不丢。
 - 提供 `compact_protect = [table_facts, last_tool_results]` 配置。

D. Tool 调用层：从"模型写 Python"升级为"调用专用 tool"

- 现状 (`nanobot/nanobot/agent/tools/`)：通用工具集（`read_file` / `exec` / `web_fetch` / `grep` 等），表格操作走 `exec` + `openpyxl` 即兴写代码。
- 限制：

- 每次操作都要模型重写 Python, token 高、易出错。
- 同样的 inspect 逻辑跑 100 次也写 100 次。
- 改造方向:
 - 首批 5 个 **tableclaw_* tool** (已在 TODO) : inspect / locate_column / aggregate / filter / topk。
 - 接入位置 `nanobot/nanobot/agent/tools/tableclaw/`, 符合 design.md 的"能力下沉到 tools/"原则。
 - tool 返回结构化结果 + 写一份缩略到 table-fact memory (4.A 联动) 。

E. Provider 层：多模型适配 + 表格专用推理预算

- 现状 (`nanobot/nanobot/providers/`) : dashscope/anthropic/openai/bedrock 等 provider 已经齐全, TableClaw 目前只绑 dashscope/deepseek。
- 限制:
 - 不同模型对表格的常识储备差异大 (deepseek-v4-pro 已知较强; claude-sonnet / gpt-4o 待测) , 结论需要多模型横比才稳。
 - reasoningEffort 当前对全任务统一 high, 简单 lookup 也开 reasoning 浪费 token。
- 改造方向:
 - 跑通 anthropic / openai 各一份 config, 做 cross-model 横评 (与 skill 横评叉乘) 。
 - 动态 **reasoning budget**: 路由层根据任务复杂度 (query keywords + 表大小) 决定 reasoning effort 档位。

4.3 多表召回检索 (mentor 提到的方向, 单独展开)

随着 dataset 扩到 80-150 题、覆盖多表场景, 模型不可能一次把所有表喂进 prompt。需要一层 retrieval:

当前缺失

- 没有"表索引"概念; 用户问"成都欠费", agent 无法主动知道哪张表里有相关数据, 只能用户指明 path。
- 多表 join / 跨期合并任务做不了——不知道哪些表应该 join。

改造方向 (与 4.A/B 共生)

层	做什么
0. 上传管道	当前 <code>workspace/uploads/</code> 尚未实装, 需要先做: 用户上传 → 落盘 → 触发索引。可在 nanobot channels 层加, 或者 webui 起来后通过 API

总	
离线索引	扫指定目录（评测期用 <code>eval_test/test_dataset/tables/</code> 或 <code>test_table/</code> ），把每张表的 (path, sheet, headers, sample rows, schema fingerprint) 入 jsonl + 一份 embedding 向量化（依赖见 4.2.B）
检索器	给定用户 query → embedding → top-k 表 + 列。可结合 BM25 + dense 双路
召回结果注入	top-k 表的 metadata 拼到 context（不是表内容，是 schema 摘要），让模型决定要读哪张
跨表 join 计划	在 retrieval 之后加一层 plan：识别需要 join 的表对（schema 重叠键）→ 推荐 join 路径

接入位置：可作为新 nanobot tool `tableclaw_search(query) -> [{table_path, sheet, why_match, schema_preview}]`，复用 4.D 的 tool 机制。

数据基础

- `workspace/uploads/<session>/` 作为用户上传区（暂未实装）
- `eval_test/test_dataset/tables/` 作为评测时的索引候选
- `test_table/` 全量池可作为压力测试

5. 三个月里程碑（建议节奏）

时间	里程碑	关键产出
第 1 周	基础设施清账	git baseline / 评分器修复 / native xlsx skill v0 / native vs codex 对照报告
第	评测扩充	

第2-3周	+ Tool RFC	dataset 到 40+ 题 / 5 个 tableclaw_* tool RFC / 多家 skill 横评 harness
第4-6周	评测到位 + Tool 落地	dataset 到 80+ 题 / 5 个 tool 实现 + A/B 报告 / 多家 skill 横评矩阵
第7-9周	系统层升级	table-aware memory v0 / skill retrieval / 多表 retrieval tool
第10-12周	商用准备	多模型 config 跑通（已存在 provider，只配文件） / 启用 nanobot 自带 OpenAI 兼容 API 并测一遍（nanobot/api/server.py 已实现） / 最简 WebUI（启用 nanobot 自带 gateway+webui） / 安全收尾（清硬编码 API key、加最简鉴权）

6. 想跟 mentor 对齐的具体问题

汇报时按这个顺序问，确保 mentor 给出最关键的方向反馈：

1. **4 大场景的优先级**：现在我们提"QA + 编辑修复 + 转换 + 报表"全场景。短期 1 个月内主推哪 1-2 个？这直接决定 native skill 拆法、dataset 题型分布、tool 选型。
2. **Dataset 协作模式**：是否同意从同学/朋友处招募贡献 20-30 题真实业务表？如果同意，是否能介绍 3-5 个潜在贡献者？
3. **多家 skill 横评是否必要**：5 家全装依赖 docker 跑成本不低（kimi linux ELF + anthropic LibreOffice）；如果 mentor 觉得"自研路线已定，不必横评"，可省下 1-2 周。
4. **系统层改造的入手点**：4.2 里 5 个子系统（memory / context / autocompact / tool / provider）+ 4.3 多表 retrieval，哪个先动？我推荐 **D (tool) → A (table-aware memory) → 4.3 (多表 retrieval)**，按"先打地基再上层"。
5. **商用形态边界**：商用是指内部公司用？SaaS 对外？还是嵌入式 SDK？决定后续 API / UI / 安全的力度。

7. 一页讲稿（5 分钟版本）

如果只有 5 分钟：

"TableClaw 是基于 nanobot 的表格专精 agent。"

现在跑通了一键启动、一键评测、一键 demo，有 10 题 skill matrix + 1 题 mentor demo 双轨迹对照，能展示 agent 在不同步骤调用不同 skill 的画面。

我们发现，在强基模下 SKILL.md 形态的边际价值有限（codex 1068 行 skill 只省 ~10% token，甚至不省）。真正的差异化要靠把 skill 下沉成专用 tool（inspect / locate / aggregate / filter / topk），并升级 nanobot 的 memory 与 context，让它对表格语义有感（记住表头结构、避免反复 inspect）。

同时 dataset 要从 11 题扩到 80-150 题，覆盖 QA / 编辑 / 转换 / 报表 4 大场景。下一步要决定的是主推哪个场景，以及系统层改造的入手点。"

附图：详见 docs/实验评测/mentor-demo/（TabelAgent.png / case.png 等）。

8. 配套材料路径速查

想看	路径
项目完整目录树	docs/架构/project-structure.md
当前 TODO + 已完成	docs/项目管理/TODO.md
开发日志（决策来龙去脉）	docs/项目管理/development-log.md
Skill 机制详解	docs/功能开发/skill-system.md
三家参考 skill 分析	docs/功能开发/reference-spreadsheet-skills.md
Token usage 设计	docs/功能开发/token-usage.md

实验线索引	<code>docs/实验评测/README.md</code>
Mentor demo 报告 + 图	<code>docs/实验评测/mentor-demo/pipeline.md</code>
Skill matrix 报告	<code>docs/实验评测/skill-matrix/xlsx-skill-selection-matrix.md</code>
一键复跑 demo	<code>./demo.sh</code>
一键复跑 10 任务评测	<code>./eval.sh</code>

9. 数据与断言的可信度

为了避免汇报时被迫问时含糊：

章节	信息来源	可信度
2 已做	实际 commit / 文件 / <code>run.json</code> / <code>latest-eval-summary.md</code> 实测	已 verify
3.1 codex skill 读 3/10、token +2.5%	<code>docs/实验评测/skill-matrix/xlsx-skill-selection-matrix.md</code> 实测数据	已 verify
3.2 mentor demo 5步/56k vs 4	<code>eval_test/results/mentor_demo/run.json</code>	已 verify

更新/迭代 步/63k	实测	反馈
4.2 nanobot 子系统现状描述	来自 nanobot/CLAUDE.md 与 docs/架构/project-structure.md 的文档级描述	文档级断言，未逐行读源码；若 mentor 追细节实现，需现场打开对应 .py 翻
4.2 改造方向	我的设计建议，未实装	方案性意见，等 mentor 拍板再设计落地
4.3 多表 retrieval	完全未实装，含上传管道、索引、检索、join 计划	方案性意见
5 三个月里程碑	建议节奏，未与 mentor 对齐	待对齐
6 想对齐的问题	我提出来的开放题	待 mentor 答

所有提到的"现状"都基于已读到的 docs/CLAUDE.md/代码产物；所有"改造方向"都明确标注为后续 TODO，没有让 mentor 误以为已实现的措辞。